Chapter 19 worked solutions • Windows 7

OS19-E01 • Granted rights and SAME_ACCESS

1. Neither hasW in its granted mask. Writable parameter storage does not grant authority. Kernel operations check the required handle rights.
2. B's0x40 is another entry or invalid;sharing produced0x88. B/internal references can retainF after A closes.
3. No. SAME_ACCESS preserves the mask. Different access depends on object/API/caller authority, including PROCESS_DUP_HANDLE for process access. Neither a raw number nor parent-child relation grants more rights. Prove the specified flag, not a universal duplication theorem.

OS19-E02 • Session scope

1. Per-process mappings, desktops/window stations, and selected scoped objects separate;kernel, drivers, physical resources, and possibly files/network remain shared. Profile location is not hardware isolation.
2. Local resolves in session namespace and can name different objects. Global may share identity but retains create/open/DACL checks. Handle numbers remain process-local.
3. No. Sessions share this kernel;VMs normally introduce guest-kernel/VMM boundaries. Driver faults/RAM pressure can affect multiple sessions. Identify scope and enforcement rather than merely different users.

OS19-E03 • Actual ReadFile outcomes

1. Pending submission is not ordinary failure. Collect through GetOverlappedResult or the chosen path. Windows 7 requires a valid non-NULL DWORD count pointer;use final completion bytes for pending operations.
2. Offset0 returns 6/ABCDEF;6 returns 4/GHIJ;10 reaches regular-file EOF, handled as appropriate zero/ERROR_HANDLE_EOF. Keep every ov/buffer/event until its terminal result and only then reuse. EOF must not spin.
3. No. Cancellation is a request. Normal completion can win, abort can yieldERROR_OPERATION_ABORTED, andNOT_FOUND still requires knowing original result. Release only after terminal;shared handles/events survive all related operations. Submission, completion, cancel statuses differ.

Chapter 19 worked solutions • Windows 7 (continued)

OS19-E04 • IRQL and thread priority

1. Neither nonzero blocking wait nor a pageable access fault is legal here. Retain context and queue waiting/pageable work to an appropriate PASSIVE worker;keep DPC short/permitted.
2. No. IRQL is processor-local. Cross-CPU access needs suitable spinlock/other synchronization with correct IRQL/order rules.
3. ISR/DPC, nonpreemptible sections, drivers, and resource waits still affect response. Without bounds/worst-case analysis, priority31 supplies no hard deadline. IRQL2 is not thread priority2.

OS19-E05 • Fixed priority queues

1. P1 runs0–1;P3 runs1–3/completes;P1 runs3–4 with residual1, leaving burst2 at tail;P2 runs4–6, leaves1;P1 runs6–8/completes;P2 runs8–9/completes. P1/P2/P3 finish8/9/3.
2. It violates immediate higher-priority preemption. A nonpreemptive/delayed-preemption model needs changed premises and recomputation.
3. No. Real quantum, boost, decay depend on environment/workload. This fixed queue model is not a measurement of every heuristic. Track residual slice and queue order.

OS19-E06 • Reserve, commit, resident

1. Reserve64 KiB, commit8 KiB, resident0. First valid touch demand-zeroes the first page, resident4 while other quantities remain. Other references/pressure are excluded.
2. Decommit removes first-page commitment/residency, leaving4 committed,0 resident,64 reserved. Release removes the reservation, all0. Actual release base/size rules still apply.
3. No. Commit promises backing resources;physical allocation can await touch. It does not immediately write pagefile pages. Equal process VAs also do not compete globally. These are different sets/statistics.

Chapter 19 worked solutions • Windows 7 (continued)

OS19-E07 • Ordered DACL

1. Remaining3→allowAlice removes1→remaining2. DenyStaff intersects2 and stops with denial;the final allow is unreachable.
2. After first allow leavesW, earlier allowStaff fills it and succeeds before deny. Order matters;canonical ordering is preferred rather than a universal deny-overrides rule.
3. NULL imposes no DACL restriction;empty exists without a granting ACE and denies this ordinary request. Other token/policy layers remain outside the simplified model. Use remaining rather than arbitrary entry union.

OS19-E08 • NTFS extents

1. VCN5, offset 123 selectsLCN201. Volume823419, disk1871995. LCN is volume-relative, so include partition base.
2. Hole returns zeros, notLCN0. Eleven logical clusters=45056 B;nine data clusters=36864 B without compression/metadata.
3. Resident means attribute content in MFT record, not RAM residency.48-bit record/16-bit sequence rejects mismatched reused references;finite sequence does not prove no wrap. Preserve layout/units.

OS19-E09 • Single-transaction WAL

1. Undo old values:bit0, map[100], size 4096. Do not retainbit1 while declaring transaction absent. Durable old records precede every allowed home change.
2. Redo all new values:bit1, map[100,200], size 8192. Required new log precedes durable commit, so home lag is recoverable.
3. No. Retain active undo/dirty redo until safe truncation;elapsed seconds alone do not authorize deletion. Metadata consistency is not all user-data or cross-file business consistency. Provide application persistence/publication. This proves the single specified transaction model, not complete NTFS concurrent recovery.

Chapter 19 worked solutions • Windows 7 (continued)

OS19-E10 • Snapshots and remote uncertainty

1. Preserve old41 before first change;S stays41 while live42 then 43. UnchangedY shares80. Reclaim backing only after all snapshots/readers release it.
2. No. Writer coordination can be required for application-consistent multi-field state. Shared storage failure can destroy both, so it is not independent backup.
3. No. Server may append before losing reply. Request identity, deduplication, idempotence, or protocol retry rules are required. TCP bytes alone do not make append exactly-once. Separate shared versions and fault domains.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter 19 worked solutions • Windows 7 (continued)

Verification code appendix: overlapped_read.cpp

```
// Self-contained Win32 example, C++17. Build: g++ -std=c++17 -municode overlapped_read.cpp
// Run with a new, nonexistent file path. The example creates and deletes only that file.
// Uses Windows7-compatible APIs; no pending buffer/OVERLAPPED/event is freed early.
#define WIN32_LEAN_AND_MEAN
#include <windows.h>
#include <array>
#include <cstdint>
#include <iostream>
#include <stdexcept>
#include <string>

struct Handle {
    HANDLE value;
    explicit Handle(HANDLE h): value(h) {}
    Handle(const Handle&) = delete;
    Handle& operator=(const Handle&) = delete;
    ~Handle() { if (value && value != INVALID_HANDLE_VALUE) CloseHandle(value); }
    bool valid() const { return value && value != INVALID_HANDLE_VALUE; }
};

[[noreturn]] void fail(const char* api) {
    DWORD code = GetLastError();
    throw std::runtime_error(std::string(api) + " error=" + std::to_string(code));
}

std::string read_at(HANDLE file, std::uint64_t offset) {
    Handle event(CreateEventW(nullptr, TRUE, FALSE, nullptr));
    if (!event.valid()) fail("CreateEventW");
    OVERLAPPED ov{};
    ov.Offset = static_cast<DWORD>(offset);
    ov.OffsetHigh = static_cast<DWORD>(offset >> 32);
    ov.hEvent = event.value;
    std::array<char, 6> buffer{};
    DWORD immediate = 0; // Windows7: ReadFile's byte-count argument must not be NULL.
    DWORD transferred = 0;
    BOOL ok = ReadFile(file, buffer.data(), static_cast<DWORD>(buffer.size()), &immediate,
&ov);
    if (!ok) {
        DWORD error = GetLastError();
        if (error == ERROR_HANDLE_EOF) return {};
        if (error != ERROR_IO_PENDING) fail("ReadFile");
    }
    // Use the operation result even if ReadFile completed immediately.
    // Wait on this operation's own event; all local storage remains alive.
```

Chapter 19 worked solutions • Windows 7 (continued)

```
    if (!GetOverlappedResult(file, &ov, &transferred, TRUE)) {
        if (GetLastError() == ERROR_HANDLE_EOF) return {};
        fail("GetOverlappedResult");
    }
    if (transferred > buffer.size()) throw std::runtime_error("invalid byte count");
    return std::string(buffer.data(), transferred);
}

int wmain(int argc, wchar_t** argv) {
    if (argc != 2) { std::cerr << "usage: demo NEW_FILE_PATH\n"; return 2; }
    bool created = false;
    try {
        {
            Handle output(CreateFileW(argv[1], GENERIC_WRITE, 0, nullptr, CREATE_NEW,
FILE_ATTRIBUTE_NORMAL, nullptr));
            if (!output.valid()) fail("CreateFileW CREATE_NEW");
            created = true;
            const std::string content = "ABCDEFGHIJ";
            DWORD done = 0;
            while (done < content.size()) {
                DWORD written = 0;
                if (!WriteFile(output.value, content.data() + done, static_cast<DWORD>
(content.size() - done), &written, nullptr)) fail("WriteFile");
                if (written == 0) throw std::runtime_error("zero-progress write");
                done += written;
            }
        }
        {
            Handle input(CreateFileW(argv[1], GENERIC_READ, FILE_SHARE_READ, nullptr,
OPEN_EXISTING, FILE_FLAG_OVERLAPPED, nullptr));
            if (!input.valid()) fail("CreateFileW OVERLAPPED");
            for (std::uint64_t offset : {0ULL, 6ULL, 10ULL}) {
                std::string data = read_at(input.value, offset);
                std::cout << data.size() << ' ' << (data.empty() ? "EOF" : data) << '\n';
            }
        }
        if (!DeleteFileW(argv[1])) fail("DeleteFileW");
        return 0;
    } catch (const std::exception& error) {
        std::cerr << error.what() << '\n';
        if (created) DeleteFileW(argv[1]);
        return 1;
    }
}
```

Chapter 19 worked solutions • Windows 7 (continued)

Verification code appendix: windows7_models.py

```
"""Self-contained deterministic Windows7 teaching models, not a kernel emulator."""
from collections import deque


def dacl_allows(sids,requested,aces):
    """Ordinary enabled-SID allow/deny model; specific toy rights only.
    None means NULL DACL; [] means empty DACL. Privileges/integrity excluded.
    """
    if not isinstance(requested,int) or not 0<=requested<=7:raise ValueError('toy
rights0..7')
    if aces is None:return True
    remaining=requested
    if not remaining:return True
    for kind,sid,mask in aces:
        if kind not in ['allow','deny'] or not 0<=mask<=7:raise ValueError('ACE')
        if sid not in sids:continue
        if kind=='deny' and remaining&mask:return False
        if kind=='allow':
            remaining&=~mask
            if remaining==0:return True
    return False


def priority_schedule():
    """Fixed priorities, quantum2, higher-priority preempt retains slice/head.
    Jobs=(release, priority, burst), ties in P1/P2/P3 order; no boosts.
    """
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 19 worked solutions • Windows 7 (continued)

```
    jobs={'P1':(0,8,4),'P2':(0,8,3),'P3':(1,10,2)}
    ready={8:deque(),10:deque()};remaining={k:j[2] for k,j in jobs.items()}
    slices={k:2 for k in jobs};current=None;timeline=[];finished={}
    for now in range(sum(remaining.values())):
        for name,(release,priority,burst) in jobs.items():
            if release==now:ready[priority].append(name)
        highest=max((p for p,q in ready.items() if q),default=-1)
        if current is not None and highest>jobs[current][1]:
            ready[jobs[current][1]].appendleft(current);current=None
        if current is None:
            priority=max(p for p,q in ready.items() if q);current=ready[priority].popleft()
        timeline.append(current);remaining[current]-=1;slices[current]-=1
        if remaining[current]==0:finished[current]=now+1;current=None
        elif slices[current]==0:
            slices[current]=2;ready[jobs[current][1]].append(current);current=None
    return timeline,finished


class PrivateRegion:
    """A fixed64KiB aligned reservation, pages4096; no shared/pinned pages here."""
    def
__init__(self):self.base=0x100000;self.reserved=16;self.committed=set();self.resident=set()
    def commit(self,page):
        if not 0<=page<self.reserved:raise ValueError('outside reservation')
        self.committed.add(page)
    def touch(self,page):
        if page not in self.committed:raise PermissionError('not committed')
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter 19 worked solutions • Windows 7 (continued)

```
        self.resident.add(page)
    def decommit(self,page):self.committed.discard(page);self.resident.discard(page)
    def release(self):self.reserved=0;self.committed.clear();self.resident.clear()
    def sizes(self):return
self.reserved*4096,len(self.committed)*4096,len(self.resident)*4096


EXTENTS=[(0,4,100),(4,3,200),(7,2,None),(9,2,190)]

def file_location(offset,extents=EXTENTS,cluster=4096,volume_base=1048576):
    if offset<0:raise ValueError('negative byte offset')
    vcn,within=divmod(offset,cluster)
    for start,length,lcn in extents:
        if start<=vcn<start+length:
            if lcn is None:return vcn,within,None,None,None
            actual=lcn+vcn-start;volume=actual*cluster+within
            return vcn,within,actual,volume,volume_base+volume
    raise EOFError('outside this file layout')


class Operation:
    """API-operation lifetime, no device work until terminal status observed."""
    def
__init__(self):self.state='new';self.cancel_requested=False;self.status=None;self.bytes=0;sel
f.freed=False
    def submit(self):
        if self.state!='new':raise ValueError('state reused')
        self.state='pending'
    def cancel(self):
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 19 worked solutions • Windows 7 (continued)

```
        if self.state!='pending':raise ValueError('not pending')
        self.cancel_requested=True
    def complete(self,status,transferred=0):
        if self.state!='pending' or status not in ['success','aborted','error'] or
transferred<0:raise ValueError('completion')
        if status=='aborted' and not self.cancel_requested:raise ValueError('model abort
needs cancel')
        self.state='terminal';self.status=status;self.bytes=transferred
    def release(self):
        if self.state!='terminal' or self.freed:raise ValueError('still referenced/already
freed')
        self.freed=True


def wal_recover(home,committed):
    """One-T complete durable old/new log. No multi-T or byte-format claim."""
    old={'size':4096,'map':(100,),'bit':0};new={'size':8192,'map':(100,200),'bit':1}
    result=dict(home);result.update(new if committed else old);return result


if __name__=='__main__':
    print('schedule',priority_schedule())
    print('file20603',file_location(20603));print('hole',file_location(7*4096+123))

r=PrivateRegion();print('reserve',r.sizes());r.commit(1);r.commit(2);print('commit',r.sizes()
);r.touch(1);print('touch',r.sizes());r.decommit(1);print('decommit',r.sizes());r.release();p
rint('release',r.sizes())
    aces=[('allow','Alice',1),('deny','Staff',2),('allow','Staff',3)]
    print('DACL requestRW',dacl_allows({'Alice','Staff'},3,aces))

op=Operation();op.submit();op.cancel();op.complete('success',6);op.release();print('cancel
raced normal completion',op.status,op.freed)
```

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study